

Real-Time British Sign Language (BSL) Translation System

For Enhanced Communication - Prototype v1.0

Project Goal: Build a working prototype that recognises selected BSL signs from raw image/video data and converts them into English text labels in **real time**.

Scope (v1.0):

- BSL alphabets (A-Z)
- Common greetings (Hello, Thank you, Yes, No) *if available in the dataset*
- Isolated sign recognition **only**

Important Disclaimer: Full BSL translation requires movement, facial expression, body posture, and sentence-level context. This prototype is limited to **isolated static sign recognition** from single images. It does **not** translate BSL grammar or continuous signing.

Dataset Used: [BSL Fingerspelling Dataset \(Kaggle\)](#)

- 20,910 images across 26 classes (A-Z)
- ~804 images per class
- ~192 MB total

Technical Stack: Python, PyTorch, Torchvision, OpenCV, MediaPipe, Pandas, NumPy, Scikit-learn, Matplotlib

Environment: Google Colab (CUDA-enabled GPU, ~12GB RAM, ~15GB VRAM)

Section 1: Setup and Installation

Install all required packages and verify CUDA availability.

```
1 import subprocess, sys
2
3 def install(pkg):
4     subprocess.check_call([sys.executable, "-m", "pip", "install", "-
5
6 packages = [
7     "torch",
8     "torchvision",
9     "opencv-python-headless",
```

```
10     "mediapipe",
11     "pandas",
12     "numpy",
13     "scikit-learn",
14     "matplotlib",
15     "seaborn",
16     "tqdm",
17     "Pillow",
18     "efficientnet-pytorch",
19     "kaggle",
20 ]
21
22 print("Installing packages...")
23 for pkg in packages:
24     install(pkg)
25 print("All packages installed!")
```

Installing packages...
All packages installed!

```
1 import os, sys, json, csv, time, copy, random, shutil, hashlib, warni
2 from pathlib import Path
3 from datetime import datetime
4 from collections import Counter, defaultdict
5
6 warnings.filterwarnings("ignore")
7
8 import numpy as np
9 import pandas as pd
10 import cv2
11 from PIL import Image
12
13 import torch
14 import torch.nn as nn
15 import torch.optim as optim
16 from torch.utils.data import Dataset, DataLoader
17 from torch.cuda.amp import GradScaler, autocast
18 import torchvision
19 from torchvision import transforms, models
20
21 import mediapipe as mp
22
23 from sklearn.model_selection import train_test_split
24 from sklearn.metrics import (
25     accuracy_score, precision_score, recall_score, f1_score,
26     classification_report, confusion_matrix
27 )
28
29 import matplotlib.pyplot as plt
30 import seaborn as sns
31 from tqdm import tqdm
32
33 print("All libraries imported!")
```

All libraries imported!

```
1 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
2
3 if torch.cuda.is_available():
4     gpu_name = torch.cuda.get_device_name(0)
5     gpu_mem = torch.cuda.get_device_properties(0).total_memory / (102
6     print(f"CUDA available! GPU: {gpu_name}, VRAM: {gpu_mem:.1f} GB")
7 else:
8     print("CUDA not available, using CPU.")
9 print(f"Device: {device}")
10
11 SEED = 42
12 random.seed(SEED); np.random.seed(SEED); torch.manual_seed(SEED)
13 if torch.cuda.is_available():
14     torch.cuda.manual_seed_all(SEED)
15     torch.backends.cudnn.deterministic = True
16     torch.backends.cudnn.benchmark = False
17 print(f"Random seed: {SEED}, Reproducibility enabled")
```

CUDA available! GPU: Tesla T4, VRAM: 14.6 GB
Device: cuda
Random seed: 42, Reproducibility enabled

✓ Section 2: Project Folder Structure

```
1 import kagglehub
2
3 # Download latest version
4 path = kagglehub.dataset_download("alifsathar/bsl-fingerspelling-data
5
6 print("Path to dataset files:", path)
```

Using Colab cache for faster access to the 'bsl-fingerspelling-dataset' da
Path to dataset files: /kaggle/input/bsl-fingerspelling-dataset

```
1 # Mount Google Drive for persistence (uncomment to use)
2 # from google.colab import drive
3 # drive.mount("/content/drive")
4
5 PROJECT_ROOT = Path("/content/BSL_Project")
6 # For Drive: PROJECT_ROOT = Path("/content/drive/MyDrive/BSL_Project")
7
8 DIRS = {
9     "raw_data": PROJECT_ROOT / "raw_data",
10    "raw_images": PROJECT_ROOT / "raw_data" / "images",
11    "raw_videos": PROJECT_ROOT / "raw_data" / "videos",
12    "processed_data": PROJECT_ROOT / "processed_data",
13    "train": Path(path) / "train",
14    "val": PROJECT_ROOT / "processed_data" / "val",
15    "test": Path(path) / "test",
```

```

16     "hand_crops": PROJECT_ROOT / "processed_data" / "hand_crops",
17     "rejected": PROJECT_ROOT / "raw_data" / "rejected",
18     "annotations": PROJECT_ROOT / "annotations",
19     "models": PROJECT_ROOT / "models",
20     "results": PROJECT_ROOT / "results",
21     "deployment": PROJECT_ROOT / "deployment",
22 }
23
24 for name, dir_path in DIRS.items():
25     dir_path.mkdir(parents=True, exist_ok=True)
26
27 print("Project structure created:")
28 for name, dir_path in sorted(DIRS.items(), key=lambda x: str(x[1])):
29     status = "OK" if dir_path.exists() else "MISSING"
30     print(f"    [{status}] {dir_path}")

```

Project structure created:

```

[OK] /content/BSL_Project/annotations
[OK] /content/BSL_Project/deployment
[OK] /content/BSL_Project/models
[OK] /content/BSL_Project/processed_data
[OK] /content/BSL_Project/processed_data/hand_crops
[OK] /content/BSL_Project/processed_data/val
[OK] /content/BSL_Project/raw_data
[OK] /content/BSL_Project/raw_data/images
[OK] /content/BSL_Project/raw_data/rejected
[OK] /content/BSL_Project/raw_data/videos
[OK] /content/BSL_Project/results
[OK] /kaggle/input/bsl-fingerspelling-dataset/test
[OK] /kaggle/input/bsl-fingerspelling-dataset/train

```

✓ Section 3: Dataset Download and Loading

Download the BSL Fingerspelling Dataset from Kaggle. You need a Kaggle API token from <https://www.kaggle.com/settings>

```

1 # The dataset has been downloaded via kagglehub in a previous step.
2 # We will use the paths already defined in our DIRS dictionary.
3
4 RAW_TRAIN_DIR = DIRS["train"]
5 RAW_TEST_DIR = DIRS["test"]
6
7 for name, path in [("Train", RAW_TRAIN_DIR), ("Test", RAW_TEST_DIR)]:
8     if path.exists():
9         print(f"Using existing {name} dataset at: {path}")
10        # List a few files to verify
11        sample_files = list(path.glob("*/*"))[:3]
12        if sample_files:
13            print(f"Found {name} data files. Ready to proceed.")
14    else:
15        print(f"Warning: {name} dataset directory not found at {path}")
16

```

```

17 # We will primarily use RAW_TRAIN_DIR for initial discovery
18 RAW_IMAGES_DIR = RAW_TRAIN_DIR

```

Using existing Train dataset at: /kaggle/input/bsl-fingerspelling-dataset/
 Found Train data files. Ready to proceed.
 Using existing Test dataset at: /kaggle/input/bsl-fingerspelling-dataset/t
 Found Test data files. Ready to proceed.

```

1 def discover_classes(data_dir):
2     data_path = Path(data_dir)
3     classes = sorted([d.name for d in data_path.iterdir() if d.is_dir]
4     class_counts = {}
5     total = 0
6     valid_ext = {".jpg", ".jpeg", ".png", ".bmp", ".gif", ".webp"}
7     for cls in classes:
8         imgs = [f for f in (data_path / cls).iterdir() if f.suffix.lower() in valid_ext]
9         class_counts[cls] = len(imgs)
10        total += len(imgs)
11    return classes, class_counts, total
12
13 classes, class_counts, total_images = discover_classes(RAW_IMAGES_DIR)
14 print(f"Classes: {len(classes)}, Total images: {total_images}")
15 print("\nClass distribution:")
16 for cls, cnt in class_counts.items():
17     bar = "#" * (cnt // 10)
18     print(f" {cls:<5}: {cnt:>5} {bar}")

```

Classes: 7, Total images: 20674

Class distribution:

```

A   : 3002 #####
B   : 3002 #####
C   : 3002 #####
D   : 3001 #####
E   : 3002 #####
F   : 2663 #####
G   : 3002 #####

```

✓ Section 4: Class Mapping and File Standardization

```

1 def create_class_mapping(classes, output_path):
2     class_mapping = {name: idx for idx, name in enumerate(classes)}
3     reverse_mapping = {idx: name for name, idx in class_mapping.items()}
4     mapping_data = {
5         "class_mapping": class_mapping,
6         "reverse_mapping": reverse_mapping,
7         "num_classes": len(classes),
8         "classes": classes,
9         "created_at": datetime.now().isoformat(),
10    }
11    out = Path(output_path) / "class_mapping.json"
12    with open(out, "w", encoding="utf-8") as f:
13        json.dump(mapping_data, f, indent=2, ensure_ascii=False)

```

```

13         json.dump(mapping_data, f, indent=2, ensure_ascii=False)
14     print(f"Class mapping saved: {out}")
15     return class_mapping, reverse_mapping, mapping_data
16
17 class_mapping, reverse_mapping, mapping_data = create_class_mapping(c
18 print("\nMapping:")
19 for name, idx in class_mapping.items():
20     print(f" {name}: {idx}")

```

Class mapping saved: /content/BSL_Project/annotations/class_mapping.json

Mapping:

```

A: 0
B: 1
C: 2
D: 3
E: 4
F: 5
G: 6

```

```

1 VALID_EXTENSIONS = {".jpg", ".jpeg", ".png", ".bmp", ".gif", ".webp"}
2
3 def standardize_filename(filepath, label_name, index):
4     ext = filepath.suffix.lower()
5     if ext not in VALID_EXTENSIONS:
6         return None
7     return f"{label_name}_user01_{index:04d}{ext}"
8
9 def standardize_dataset(data_dir, class_mapping):
10     """
11     NOTE: This function is skipped because /kaggle/input is read-only
12     Standardization will be handled during the 'Organize Dataset' step
13     where files are copied to a writable directory.
14     """
15     print("Skipping in-place standardization: Source directory is read-only")
16     print("File names will be managed during the organization/copying")
17
18 # standardize_dataset(RAW_IMAGES_DIR, class_mapping)

```

✓ Section 5: Raw Data Validation

```

1 import hashlib
2 from pathlib import Path
3 import pandas as pd
4 import shutil
5 from PIL import Image
6 import cv2
7 import numpy as np
8 from tqdm import tqdm
9 import os
10
11 def compute_file_hash(filepath, block_size=65536):
12     h = hashlib.sha256()

```

```
12     n = hashlib.md5()
13     try:
14         with open(filepath, "rb") as f:
15             for block in iter(lambda: f.read(block_size), b''):
16                 h.update(block)
17             return h.hexdigest()
18     except Exception:
19         return None
20
21 def validate_image(filepath):
22     path = Path(filepath)
23
24     # Check file exists and get stats
25     if not path.exists():
26         return False, "File does not exist"
27
28     try:
29         stat_info = path.stat()
30     except OSError:
31         return False, "Cannot access file"
32
33     if stat_info.st_size == 0:
34         return False, "Empty file"
35
36     if stat_info.st_size < 1024: # Less than 1KB
37         return False, "File too small"
38
39     # Validate with PIL
40     try:
41         with Image.open(filepath) as img:
42             # Don't use verify() method as it doesn't always work co
43             # Instead, try to get basic properties
44             img_format = img.format
45             img_size = img.size
46             img_mode = img.mode
47
48             # Basic checks
49             if img_size[0] < 10 or img_size[1] < 10:
50                 return False, f"Image too small: {img_size}"
51
52             # Try to convert to RGB to ensure it's a valid image
53             if img.mode in ('RGBA', 'LA', 'P'):
54                 img = img.convert('RGB')
55             elif img.mode != 'RGB':
56                 img = img.convert('RGB')
57
58     except Exception as e:
59         return False, f"PIL error: {str(e)[:80]}"
60
61     # Validate with OpenCV
62     try:
63         img_cv = cv2.imread(str(filepath))
64         if img_cv is None:
65             return False, "OpenCV could not read"
```

```

66
67     height, width = img_cv.shape[:2]
68     if width < 10 or height < 10:
69         return False, f"OpenCV: Image too small: ({width}, {height})"
70
71 except Exception as e:
72     return False, f"OpenCV error: {str(e)[:80]}"
73
74 return True, "OK"
75
76 def validate_dataset(data_dir, rejected_dir=None):
77     data_path = Path(data_dir)
78
79     # Only create rejected directory if specified and we want to move
80     if rejected_dir:
81         rejected_path = Path(rejected_dir)
82         rejected_path.mkdir(parents=True, exist_ok=True)
83     else:
84         rejected_path = None
85
86     results = []
87     seen_hashes = {}
88     total = 0
89     valid = 0
90     rejected = 0
91
92     all_images = []
93     for cls_dir in sorted(data_path.iterdir()):
94         if cls_dir.is_dir():
95             for f in cls_dir.iterdir():
96                 if f.suffix.lower() in ['.jpg', '.jpeg', '.png', '.bmp']:
97                     all_images.append(f)
98
99     print(f"Validating {len(all_images)} images...")
100
101     for filepath in tqdm(all_images, desc="Validating"):
102         total += 1
103         label = filepath.parent.name
104
105         # Compute file hash
106         fh = compute_file_hash(filepath)
107
108         # Check for duplicates
109         if fh and fh in seen_hashes:
110             results.append({
111                 "image_path": str(filepath),
112                 "label": label,
113                 "status": "rejected",
114                 "reason": f"Duplicate of {seen_hashes[fh]}"
115             })
116             rejected += 1
117             continue # Skip moving since input dir is read-only
118
119         if fh:
120             seen_hashes[fh] = filepath.name

```



```

120         seen_hashes[img] = filepath.name
121
122     # Validate the image
123     try:
124         ok, reason = validate_image(filepath)
125     except Exception as e:
126         ok = False
127         reason = f"Unexpected error during validation: {str(e)}"
128
129     if ok:
130         results.append({
131             "image_path": str(filepath),
132             "label": label,
133             "status": "valid",
134             "reason": "OK"
135         })
136         valid += 1
137     else:
138         results.append({
139             "image_path": str(filepath),
140             "label": label,
141             "status": "rejected",
142             "reason": reason
143         })
144         rejected += 1
145
146     # Create results DataFrame and save
147     results_df = pd.DataFrame(results)
148
149     # Only save to annotations if DIRS is defined, otherwise save to
150     try:
151         results_df.to_csv(DIRS["annotations"] / "validation_log.csv"
152     except:
153         results_df.to_csv("validation_log.csv", index=False)
154
155     print(f"Validation complete: Valid: {valid}, Rejected: {rejected}
156
157     # Print summary
158     rejected_by_reason = results_df[results_df['status'] == 'rejecte
159     if len(rejected_by_reason) > 0:
160         print("\nRejected by reason:")
161         for reason, count in rejected_by_reason.items():
162             print(f" {reason}: {count}")
163
164     return results_df
165
166 # Define common image extensions if not already defined
167 VALID_EXTENSIONS = {'.jpg', '.jpeg', '.png', '.bmp', '.tiff', '.tif'
168
169 # Run validation (note: we don't specify a rejected_dir since input
170 validation_df = validate_dataset(RAW_IMAGES_DIR)

```

Validating 20674 images...

Validating: 100%██████████ 20674/20674 [01:37<00:00, 212.32it/s]

Validation complete: Valid: 18428, Rejected: 2246, Total: 20674

Rejected by reason:

Duplicate of F138.jpg: 1
Duplicate of A603 flip.jpg: 1
Duplicate of A628.jpg: 1
Duplicate of A148.jpg: 1
Duplicate of A167.jpg: 1
Duplicate of A800.jpg: 1
Duplicate of F219.jpg: 1
Duplicate of F128 flip.jpg: 1
Duplicate of F92 flip.jpg: 1
Duplicate of F142 flip.jpg: 1
Duplicate of F61 flip.jpg: 1
Duplicate of F177.jpg: 1
Duplicate of F294 flip.jpg: 1
Duplicate of F143 flip.jpg: 1
Duplicate of F86 flip.jpg: 1
Duplicate of F52.jpg: 1
Duplicate of F96.jpg: 1
Duplicate of F33 flip.jpg: 1
Duplicate of F252 flip.jpg: 1
Duplicate of F83 flip.jpg: 1
Duplicate of F221.jpg: 1
Duplicate of F157 flip.jpg: 1
Duplicate of F217 flip.jpg: 1
Duplicate of F73 flip.jpg: 1
Duplicate of F67 flip.jpg: 1
Duplicate of F282.jpg: 1
Duplicate of F193 flip.jpg: 1
Duplicate of F118.jpg: 1
Duplicate of F93 flip.jpg: 1
Duplicate of F99 flip.jpg: 1
Duplicate of F276 flip.jpg: 1
Duplicate of F280.jpg: 1
Duplicate of F66 flip.jpg: 1
Duplicate of F130 flip.jpg: 1
Duplicate of F83.jpg: 1
Duplicate of F160.jpg: 1
Duplicate of F126.jpg: 1
Duplicate of F578.jpg: 1
Duplicate of A741 flip.jpg: 1
Duplicate of A612.jpg: 1
Duplicate of A834 flip.jpg: 1
Duplicate of A300 flip.jpg: 1
Duplicate of A732.jpg: 1
Duplicate of A18 flip.jpg: 1
Duplicate of A684 flip.jpg: 1
Duplicate of A373 flip.jpg: 1
Duplicate of A289 flip.jpg: 1
Duplicate of A51 flip.jpg: 1
Duplicate of A193 flip.jpg: 1
Duplicate of A386.jpg: 1
Duplicate of A327 flip.jpg: 1
Duplicate of A21 flip.jpg: 1
Duplicate of A704.jpg: 1

✓ Section 6: Video Frame Extraction

If the dataset contains video files, extract frames using OpenCV VideoCapture. The BSL Fingerspelling Dataset is image-only, but this section handles video data for extensibility.

```

1 RAW_VIDEOS_DIR = DIRS["raw_videos"]
2 VIDEO_FRAMES_DIR = PROJECT_ROOT / "raw_data" / "video_frames"
3 VIDEO_FRAMES_DIR.mkdir(parents=True, exist_ok=True)
4
5 def extract_frames_from_videos(videos_dir, output_dir, frame_interval
6     vp = Path(videos_dir); op = Path(output_dir); op.mkdir(parents=Tr
7     vid_ext = {".mp4", ".avi", ".mov", ".mkv", ".wmv", ".flv", ".webm
8     vids = [f for f in vp.iterdir() if f.suffix.lower() in vid_ext]
9     if not vids:
10         print("No video files found. Skipping frame extraction.")
11         return []
12     log = []
13     print(f"Found {len(vids)} video(s)")
14     for vf in tqdm(vids, desc="Extracting"):
15         label = vf.parent.name if vf.parent.name != str(vp) else vf.s
16         label_dir = op / label; label_dir.mkdir(parents=True, exist_o
17         cap = cv2.VideoCapture(str(vf))
18         if not cap.isOpened():
19             print(f"Cannot open: {vf.name}"); continue
20         fc = 0; extracted = 0
21         while extracted < max_frames:
22             ret, frame = cap.read()
23             if not ret: break
24             if fc % frame_interval == 0:
25                 fname = f"{label}_{vf.stem}_{extracted:04d}.jpg"
26                 cv2.imwrite(str(label_dir / fname), frame)
27                 log.append({"video": str(vf), "frame": fname, "label"
28                 extracted += 1
29                 fc += 1
30             cap.release()
31     if log:
32         pd.DataFrame(log).to_csv(DIRS["annotations"] / "video_extract
33         print(f"Extracted {len(log)} frames")
34     return log
35
36 video_log = extract_frames_from_videos(RAW_VIDEOS_DIR, VIDEO_FRAMES_D

```

No video files found. Skipping frame extraction.

✓ Section 7: Manual Label Annotation

```

1 def generate_annotations(train_dir, test_dir, class_mapping, output_p
2     annotations = []
3     for data_dir, split_label in [(train_dir, "train"), (test_dir, "t
4         data_path = Path(data_dir)

```

```

5     all_imgs = []
6     for cls_dir in sorted(data_path.iterdir()):
7         if cls_dir.is_dir():
8             for f in sorted(cls_dir.iterdir()):
9                 if f.suffix.lower() in VALID_EXTENSIONS:
10                     all_imgs.append(f)
11
12     print(f"Generating annotations for {len(all_imgs)} images in
13     for fp in tqdm(all_imgs, desc=f"Annotating {split_label}"):
14         label_name = fp.parent.name
15         label_id = class_mapping.get(label_name, -1)
16         parts = fp.stem.split("_")
17         user_id = next((p for p in parts if p.startswith("user")))
18         annotations.append({
19             "image_path": str(fp), "label_name": label_name,
20             "label_id": label_id, "user_id": user_id,
21             "split": split_label, "quality_status": "pending", "n
22         })
23
24     df = pd.DataFrame(annotations)
25     df.to_csv(output_path, index=False)
26     print(f"\nTotal annotations saved: {output_path} ({len(df)} entri
27     return df
28
29 annotations_df = generate_annotations(RAW_TRAIN_DIR, RAW_TEST_DIR, cl
30 print("\nPreview:")
31 print(annotations_df.head(5).to_string())

```

```

Generating annotations for 20674 images in train...
Annotating train: 100%|██████████| 20674/20674 [00:00<00:00, 151024.08it/s]
Generating annotations for 236 images in test...
Annotating test: 100%|██████████| 236/236 [00:00<00:00, 84798.74it/s]

```

```
Total annotations saved: /content/BSL_Project/annotations/annotations.csv
```

```
Preview:
```

	image_path	label_name
0	/kaggle/input/bsl-fingerspelling-dataset/train/A/A0 flip.jpg	
1	/kaggle/input/bsl-fingerspelling-dataset/train/A/A0.jpg	
2	/kaggle/input/bsl-fingerspelling-dataset/train/A/A1 flip.jpg	
3	/kaggle/input/bsl-fingerspelling-dataset/train/A/A1.jpg	
4	/kaggle/input/bsl-fingerspelling-dataset/train/A/A10 flip.jpg	

Section 8: MediaPipe Hand Annotation

Use MediaPipe Hands to detect hand regions, extract bounding boxes and landmarks, and create cropped hand images.

```

1 import os
2 import urllib.request
3 import mediapipe as mp
4 from mediapipe.tasks import python

```

```

5 from mediapipe.tasks.python import vision
6
7 # 1. Download the Hand Landmarker model file
8 model_path = 'hand_landmarker.task'
9 if not os.path.exists(model_path):
10     print("Downloading Hand Landmarker model...")
11     url = "https://storage.googleapis.com/mediapipe-models/hand_landm
12     urllib.request.urlretrieve(url, model_path)
13
14 # 2. Initialize the Hand Landmarker using the Tasks API
15 base_options = python.BaseOptions(model_asset_path=model_path)
16 options = vision.HandLandmarkerOptions(
17     base_options=base_options,
18     num_hands=2,
19     min_hand_detection_confidence=0.5,
20     min_hand_presence_confidence=0.5,
21     min_tracking_confidence=0.5
22 )
23 hands_detector = vision.HandLandmarker.create_from_options(options)
24
25 print(f"MediaPipe Tasks HandLandmarker initialized successfully using

```

Downloading Hand Landmarker model...

MediaPipe Tasks HandLandmarker initialized successfully using hand_landmar

```

1 HAND_CROPS_DIR = DIRS["hand_crops"]
2
3 def process_image_mediapipe(image_path, detector, crop_dir):
4     p = Path(image_path)
5     result = {"image_path": str(p), "hand_detected": False, "num_hands": 0,
6               "x_min": "", "y_min": "", "x_max": "", "y_max": "",
7               "landmark_points": "", "crop_path": "", "review_flag": ""}
8
9     img_bgr = cv2.imread(str(p))
10    if img_bgr is None:
11        result["review_flag"] = "unreadable_image"; return result
12
13    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
14    h, w = img_rgb.shape[:2]
15
16    # Convert to MediaPipe Image object for Tasks API
17    mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=img_rgb)
18
19    # Detect landmarks
20    detection_result = detector.detect(mp_image)
21
22    if detection_result.hand_landmarks:
23        result["hand_detected"] = True
24        result["num_hands"] = len(detection_result.hand_landmarks)
25
26        # Use first detected hand
27        hl = detection_result.hand_landmarks[0]
28        lm_list, xc, yc = [], [], []
29

```

```

29
30     for lm in hl:
31         xc.append(lm.x * w); yc.append(lm.y * h)
32         lm_list.append([round(lm.x * w, 2), round(lm.y * h, 2)])
33
34     x1 = max(0, int(min(xc)) - 20); y1 = max(0, int(min(yc)) - 20)
35     x2 = min(w, int(max(xc)) + 20); y2 = min(h, int(max(yc)) + 20)
36
37     if (x2 - x1) < 30 or (y2 - y1) < 30:
38         result["review_flag"] = "bounding_box_too_small"
39         cx, cy = (x1 + x2) // 2, (y1 + y2) // 2
40         x1 = max(0, cx - 50); x2 = min(w, cx + 50)
41         y1 = max(0, cy - 50); y2 = min(h, cy + 50)
42
43     result.update({"x_min": x1, "y_min": y1, "x_max": x2, "y_max":
44                  "landmark_points": json.dumps(lm_list)})
45
46     crop = img_rgb[y1:y2, x1:x2]
47     label = p.parent.name
48     cdir = Path(crop_dir) / label; cdir.mkdir(parents=True, exist_
49     cpath = cdir / f"crop_{p.stem}.jpg"
50     cv2.imwrite(str(cpath), cv2.cvtColor(crop, cv2.COLOR_RGB2BGR))
51     result["crop_path"] = str(cpath)
52
53     if result["num_hands"] > 1:
54         result["review_flag"] = ("multiple_hands" if not result["r
55 else:
56     result["review_flag"] = "no_hand_detected"
57 return result
58
59 def annotate_all_images(ann_df, detector, crop_dir):
60     print(f"Processing {len(ann_df)} images with MediaPipe Tasks API..
61     results = []; stats = Counter()
62     # Processing a sample subset for speed in this prototype step if n
63     for _, row in tqdm(ann_df.iterrows(), total=len(ann_df), desc="Med
64         r = process_image_mediapipe(row["image_path"], detector, crop_
65         r["label_name"] = row["label_name"]; r["label_id"] = row["labe
66         results.append(r)
67         stats["hand_detected" if r["hand_detected"] else "no_hand"] +=
68         if r["review_flag"]: stats["needs_review"] += 1
69
70     df = pd.DataFrame(results)
71     df.to_csv(DIRS["annotations"] / "hand_annotations.csv", index=False
72     print(f"Done. Hand detected: {stats['hand_detected']}, No hand: {s
73     return df
74
75 print("Beginning hand detection processing...")
76 hand_annotations_df = annotate_all_images(annotations_df, hands_detect

```

```

Beginning hand detection processing...
Processing 20910 images with MediaPipe Tasks API...
MediaPipe: 100%|██████████| 20910/20910 [11:24<00:00, 30.55it/s]
Done. Hand detected: 12422, No hand: 8488, Needs review: 11862

```

Section 9: Manual Review of Automatic Annotations

```

1 def review_annotations(hand_df):
2     flagged = hand_df[hand_df["review_flag"] != ""].copy()
3     if len(flagged) == 0:
4         print("No images flagged for review!"); return flagged
5     print(f"{len(flagged)} images flagged:")
6     for flag_type in ["no_hand_detected", "bounding_box_too_small", "
7         subset = flagged[flagged["review_flag"].str.contains(flag_type)
8         if len(subset) > 0:
9             print(f"  {flag_type}: {len(subset)}")
10            for _, r in subset.head(2).iterrows():
11                print(f"    - {Path(r['image_path']).name} ({r['label']})
12    flagged.to_csv(DIRS["annotations"] / "review_report.csv", index=False)
13    return flagged
14
15 review_df = review_annotations(hand_annotations_df)

```

```

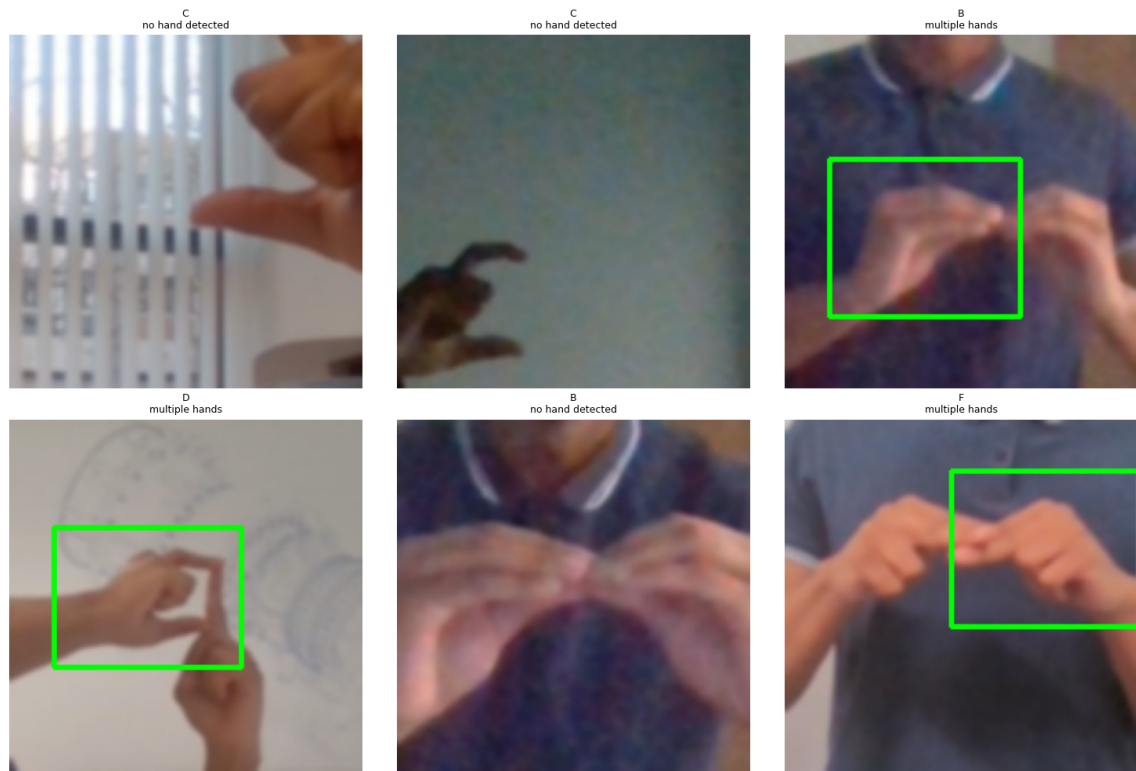
11862 images flagged:
  no_hand_detected: 8488
    - B108 flip.jpg (B)
    - B1208 flip.jpg (B)
  multiple_hands: 3374
    - A1055.jpg (A)
    - A1058 flip.jpg (A)

```

```

1 def visualize_flagged(hand_df, max_samples=6):
2     flagged = hand_df[hand_df["review_flag"] != ""]
3     if len(flagged) == 0:
4         print("No flagged images."); return
5     samples = flagged.sample(n=min(max_samples, len(flagged)), random=True)
6     ncols = 3; nrows = (len(samples) + ncols - 1) // ncols
7     fig, axes = plt.subplots(nrows, ncols, figsize=(15, 5 * nrows))
8     axes = axes.flatten() if nrows > 1 else [axes]
9     for idx, (_, row) in enumerate(samples.iterrows()):
10        ax = axes[idx]
11        img = cv2.imread(row["image_path"])
12        if img is not None:
13            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
14            if row["x_min"] != "":
15                cv2.rectangle(img, (int(row["x_min"]), int(row["y_min"])),
16                            (int(row["x_max"]), int(row["y_max"])),
17                            (0, 0, 0))
18            ax.imshow(img)
19            ax.set_title(f"{row['label_name']}\n{row['review_flag']}")
20            ax.axis("off")
21        for i in range(len(samples), len(axes)): axes[i].axis("off")
22    plt.tight_layout()
23    plt.savefig(DIRS["results"] / "flagged_images_review.png", dpi=15)
24    plt.show()
25    print("Saved: results/flagged_images_review.png")
26 visualize_flagged(hand_annotations_df)

```



Saved: results/flagged_images_review.png

✓ Section 10: Final Annotations

```

1 def create_final_annotations(ann_df, hand_df, output_path):
2     merged = ann_df.merge(
3         hand_df[[ "image path", "hand detected", "num hands",

```



```

4         "x_min", "y_min", "x_max", "y_max",
5         "landmark_points", "crop_path", "review_flag"]],
6         on="image_path", how="left",
7     )
8     def get_quality(row):
9         flag = str(row.get("review_flag", ""))
10        if "unreadable" in flag: return "rejected"
11        if not row.get("hand_detected", False): return "no_hand_detected"
12        if "too_small" in flag: return "poor_crop"
13        if "multiple" in flag: return "multiple_hands"
14        if row.get("hand_detected", False): return "good"
15        return "pending"
16    merged["quality_status"] = merged.apply(get_quality, axis=1)
17    merged = merged.sort_values(["label_name", "image_path"]).reset_index()
18    merged.to_csv(output_path, index=False)
19    print(f"Final annotations: {output_path} ({len(merged)} entries)")
20    print("Quality distribution:")
21    for s, c in merged["quality_status"].value_counts().items():
22        print(f"    {s}: {c} ({100*c/len(merged):.1f}%)")
23    return merged
24
25 final_ann_path = DIRS["annotations"] / "final_annotations.csv"
26 final_df = create_final_annotations(annotations_df, hand_annotations_df)
27 print("\nPreview:")
28 print(final_df.head().to_string())

```

```

Final annotations: /content/BSL_Project/annotations/final_annotations.csv
Quality distribution:
good: 9048 (43.3%)
no_hand_detected: 8488 (40.6%)
multiple_hands: 3374 (16.1%)

```

Preview:

```

0         /kaggle/input/bsl-fingerspelling-dataset/test/A/A
1  /kaggle/input/bsl-fingerspelling-dataset/test/A/A (access ambassadors)
2  /kaggle/input/bsl-fingerspelling-dataset/test/A/A (access ambassadors)
3         /kaggle/input/bsl-fingerspelling-dataset/test/A/A
4         /kaggle/input/bsl-fingerspelling-dataset/test/A/A

```

✓ Section 11: Dataset Splitting

Stratified split into train (75%), validation (12.5%), and test (12.5%). Prevents user leakage where possible.

```

1 def stratified_split(df, label_col="label_name", user_col="user_id",
2                       train_r=0.75, val_r=0.125, test_r=0.125, seed=SEED)
3     # CRITICAL FIX: Only keep rows where label_id is valid (>= 0)
4     valid = df[(df[label_col].isin(["good", "no_hand_detected", "multiple_hands"])
5               & df[user_col].isin(["good", "no_hand_detected", "multiple_hands"])
6               & df[label_col] != "unreadable"]
7     print(f"Splitting {len(valid)} valid images with correct labels..")
8     # Split into train, validation, and test sets
9     rng = random.Random(seed)
10    users = valid[user_col].unique()
11    # Shuffle users
12    rng.shuffle(users)
13    # Split users into train, validation, and test sets
14    train_users = users[:int(len(users)*train_r)]
15    val_users = users[int(len(users)*train_r):int(len(users)*(train_r+val_r))]
16    test_users = users[int(len(users)*(train_r+val_r)):]
17    # Split data into train, validation, and test sets
18    train_df = valid[valid[user_col].isin(train_users)]
19    val_df = valid[valid[user_col].isin(val_users)]
20    test_df = valid[valid[user_col].isin(test_users)]
21    return train_df, val_df, test_df

```

```

7     has_user = valid[user_col].nunique() > 1 and unknown not in val
8     if has_user:
9         user_lbl = valid.groupby(user_col)[label_col].agg(lambda x: C
10         ulist = list(user_lbl.index)
11         utrain, utest = train_test_split(ulist, test_size=(val_r + te
12         uval, utest2 = train_test_split(utest, test_size=test_r / (va
13         train_df = valid[valid[user_col].isin(utrain)]
14         val_df = valid[valid[user_col].isin(uval)]
15         test_df = valid[valid[user_col].isin(utest2)]
16     else:
17         train_df, temp = train_test_split(valid, test_size=(val_r + t
18         val_df, test_df = train_test_split(temp, test_size=test_r / (
19
20     train_df = train_df.copy(); val_df = val_df.copy(); test_df = tes
21     train_df["split"] = "train"; val_df["split"] = "val"; test_df["sp
22
23     result = df.copy()
24     result["split"] = "excluded" # Default everything to excluded
25     for sdf in [train_df, val_df, test_df]:
26         mask = result["image_path"].isin(sdf["image_path"])
27         result.loc[mask, "split"] = sdf["split"].iloc[0]
28
29     print(f"\nSplit complete (Valid Classes Only):")
30     for name, sdf in [("Train", train_df), ("Val", val_df), ("Test",
31         print(f" {name}: {len(sdf)} images")
32     return result, train_df, val_df, test_df
33
34 final_df, train_df, val_df, test_df = stratified_split(final_df)
35 final_df.to_csv(final_ann_path, index=False)

```

Splitting 20742 valid images with correct labels...

Split complete (Valid Classes Only):

Train: 15556 images

Val: 2593 images

Test: 2593 images

✓ Section 12: Organize Processed Dataset

```

1 def organize_dataset(final_df, base_dir, use_crops=True):
2     base = Path(base_dir); organized = 0; fallback = 0; missing = 0
3     for split_name in ["train", "val", "test"]:
4         sdf = final_df[final_df["split"] == split_name]
5         if len(sdf) == 0: continue
6         print(f" Organizing {split_name} ({len(sdf)} images)...")
7         for _, row in tqdm(sdf.iterrows(), total=len(sdf), desc=f" {
8             label = row["label_name"]
9             sdir = base / split_name / label; sdir.mkdir(parents=True
10             src = None
11             if use_crops and row.get("hand_detected") and row.get("cr
12                 cp = Path(row["crop_path"])
13                 if cp.exists(): src = cp
14             else: src = Path(row["image path"]) if Path(row["imag

```

```

15         else:
16             src = Path(row["image_path"]) if Path(row["image_path"]
17             if src is None: missing += 1; continue
18             dest = sdir / src.name
19             if not dest.exists(): shutil.copy2(str(src), str(dest))
20             organized += 1
21     print(f"\nOrganized: {organized}, Fallback: {fallback}, Missing:
22     for s in ["train", "val", "test"]:
23         sp = base / s
24         if sp.exists():
25             nc = len([d for d in sp.iterdir() if d.is_dir()])
26             ni = sum(len(list(d.iterdir())) for d in sp.iterdir() if
27             print(f"  {s}: {nc} classes, {ni} images")
28
29 print("Organizing dataset (using hand crops where available)...")
30 organize_dataset(final_df, DIRS["processed_data"], use_crops=True)

```

```

Organizing dataset (using hand crops where available)...
Organizing train (15556 images)...
train: 100%|██████████| 15556/15556 [00:10<00:00, 1450.10it/s]
Organizing val (2593 images)...
val: 100%|██████████| 2593/2593 [00:01<00:00, 1494.16it/s]
Organizing test (2593 images)...
test: 100%|██████████| 2593/2593 [00:01<00:00, 1549.22it/s]
Organized: 20742, Fallback: 8467, Missing: 0
train: 7 classes, 15556 images
val: 7 classes, 2593 images
test: 7 classes, 2593 images

```

✓ Section 13: Preprocessing and Data Augmentation

```

1 IMAGENET_MEAN = [0.485, 0.456, 0.406]
2 IMAGENET_STD = [0.229, 0.224, 0.225]
3 IMAGE_SIZE = 224
4
5 train_transform = transforms.Compose([
6     transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
7     transforms.RandomRotation(degrees=10),
8     transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0
9     transforms.RandomResizedCrop(IMAGE_SIZE, scale=(0.9, 1.0), ratio=
10     transforms.RandomAffine(degrees=0, translate=(0.05, 0.05)),
11     transforms.ToTensor(),
12     transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
13 ])
14
15 val_transform = transforms.Compose([
16     transforms.Resize((IMAGE_SIZE, IMAGE_SIZE)),
17     transforms.ToTensor(),
18     transforms.Normalize(mean=IMAGENET_MEAN, std=IMAGENET_STD),
19 ])
20
21 print(f"Image size: {IMAGE_SIZE}x{IMAGE_SIZE}")

```

```

21 print('Image size: (%d,%d)' % (image_size, image_size))
22 print("Train augmentation: rotation(10), color jitter, random crop, t
23 print("Note: Aggressive flipping NOT used (could change sign meaning)

```

Image size: 224x224

Train augmentation: rotation(10), color jitter, random crop, translation

Note: Aggressive flipping NOT used (could change sign meaning)

✓ Section 14: Custom PyTorch Dataset

Memory-efficient dataset that loads images on-the-fly. Implements `__init__`, `__len__`, `__getitem__`.

```

1 class BSLSignDataset(Dataset):
2     def __init__(self, annotations_file, split, transform=None, mode=
3         self.df = pd.read_csv(annotations_file)
4         self.df = self.df[self.df['split'] == split].reset_index(drop
5         self.transform = transform
6         self.mode = mode
7
8     def __len__(self):
9         return len(self.df)
10
11     def __getitem__(self, idx):
12         row = self.df.iloc[idx]
13         img_path = row['image_path']
14
15         # Use hand crop if available and requested
16         if self.mode == "auto" and pd.notna(row['crop_path']) and os.
17             img_path = row['crop_path']
18
19         image = Image.open(img_path).convert("RGB")
20         label = int(row['label_id'])
21
22         if self.transform:
23             image = self.transform(image)
24
25         return image, label
26
27 print("Re-creating datasets and loaders...")
28 train_dataset = BSLSignDataset(final_ann_path, "train", train_transfo
29 val_dataset = BSLSignDataset(final_ann_path, "val", val_transform, "a
30 test_dataset = BSLSignDataset(final_ann_path, "test", val_transform,
31
32 # Define loader parameters here to avoid NameError
33 BATCH_SIZE = 32
34 NUM_WORKERS = 2
35 PIN_MEM = torch.cuda.is_available()
36
37 train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuff
38 val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=F
39 test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle=

```

```

39 test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle
40
41 print(f"Created loaders with BATCH_SIZE={BATCH_SIZE}")

```

Re-creating datasets and loaders...
Created loaders with BATCH_SIZE=32

✓ Section 15: PyTorch DataLoaders

```

1 BATCH_SIZE = 32; NUM_WORKERS = 2; PIN_MEM = torch.cuda.is_available()
2
3 train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle
4                             num_workers=NUM_WORKERS, pin_memory=PIN_MEM
5 val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=F
6                             num_workers=NUM_WORKERS, pin_memory=PIN_MEM)
7 test_loader = DataLoader(test_dataset, batch_size=BATCH_SIZE, shuffle
8                             num_workers=NUM_WORKERS, pin_memory=PIN_MEM)
9
10 print(f"Batch size: {BATCH_SIZE}, Workers: {NUM_WORKERS}, Pin: {PIN_M
11 print(f"Train batches: {len(train_loader)}, Val: {len(val_loader)}, T
12
13 imgs, lbls = next(iter(train_loader))
14 print(f"Sample batch: {imgs.shape}, labels: {lbls.shape}")
15 print(f"Tensor range: [{imgs.min():.3f}, {imgs.max():.3f}]")

```

Batch size: 32, Workers: 2, Pin: True
Train batches: 486, Val: 82, Test: 82
Sample batch: torch.Size([32, 3, 224, 224]), labels: torch.Size([32])
Tensor range: [-2.118, 2.483]

✓ Section 16: CNN Model Creation (Transfer Learning)

Default: EfficientNet-B0 pretrained on ImageNet. Alternatives: MobileNetV3-Small, ResNet-18.

```

1 from efficientnet_pytorch import EfficientNet
2
3 # Ensure NUM_CLASSES strictly matches the current classes list
4 NUM_CLASSES = len(classes)
5 print(f"Initializing models for {NUM_CLASSES} classes: {classes}")
6
7 def create_efficientnet_b0(num_classes, pretrained=True):
8     # Using EfficientNet-B0 as the primary architecture
9     model = EfficientNet.from_pretrained("efficientnet-b0" if pretrai
10                                         num_classes=num_classes)
11     return model
12
13 def create_mobilenet_v3(num_classes, pretrained=True):
14     w = models.MobileNet_V3_Small_Weights.IMAGENET1K_V1 if pretrained
15     model = models.mobilenet_v3_small(weights=w)

```

```

15     m = models.mobilenet_v3_small(weights=w)
16     m.classifier = nn.Sequential(
17         nn.Linear(576, 256), nn.Hardswish(inplace=True),
18         nn.Dropout(p=0.2), nn.Linear(256, num_classes))
19     return m
20
21 def create_resnet18(num_classes, pretrained=True):
22     w = models.ResNet18_Weights.IMAGENET1K_V1 if pretrained else None
23     m = models.resnet18(weights=w)
24     m.fc = nn.Linear(m.fc.in_features, num_classes)
25     return m
26
27 print("Creating EfficientNet-B0...")
28 try:
29     # Move model to device
30     model = create_efficientnet_b0(NUM_CLASSES, pretrained=True).to(d
31
32     total_p = sum(p.numel() for p in model.parameters())
33     train_p = sum(p.numel() for p in model.parameters() if p.requires_
34     print(f"Parameters: {total_p:,} total, {train_p:,} trainable")
35     print(f"Model size: ~{total_p * 4 / (1024**2):.1f} MB")
36     print(f"Alternatives available: create_mobilenet_v3, create_resne
37 except Exception as e:
38     print(f"Error initializing model: {e}")
39     print("Tip: If a CUDA error persists, try 'Runtime -> Restart ses

```

```

Initializing models for 7 classes: ['A', 'B', 'C', 'D', 'E', 'F', 'G']
Creating EfficientNet-B0...
Downloading: "https://github.com/lukemelas/EfficientNet-PyTorch/releases/d
100%|██████████| 20.4M/20.4M [00:00<00:00, 247MB/s]Loaded pretrained weigh
Parameters: 4,016,515 total, 4,016,515 trainable
Model size: ~15.3 MB
Alternatives available: create_mobilenet_v3, create_resnet18

```

✓ Section 17: Training Configuration

```

1 def compute_class_weights(dataset):
2     # Extract all labels from the dataset
3     labels = [dataset.df.iloc[i]["label_id"] for i in range(len(datas
4     counts = Counter(labels)
5     total = len(labels)
6
7     # We must ensure weights are provided for every class index up to
8     # to prevent indexing errors in the loss function.
9     n_cls = max(counts.keys()) + 1 if counts else 0
10
11     weights = []
12     for c in range(n_cls):
13         count = counts.get(c, 0)
14         if count > 0:
15             # Inverse frequency: total / (number_of_classes * class_c
16             weights.append(total / (n_cls * count))

```

```

17         else:
18             # Placeholder for missing classes to maintain tensor shape
19             weights.append(1.0)
20
21     return torch.tensor(weights, dtype=torch.float32).to(device)
22
23 try:
24     class_weights = compute_class_weights(train_dataset)
25     print(f"Class weights computed for {len(class_weights)} classes.")
26 except Exception as e:
27     print(f"Error computing weights: {e}")
28     print("TIP: If you see a 'CUDA error', please Restart Session (Ru

```

Class weights computed for 7 classes.

```

1 CONFIG = {
2     "epochs": 10,
3     "learning_rate": 3e-4,
4     "weight_decay": 1e-4,
5     "patience": 5,
6     "min_delta": 0.001,
7     "confidence_threshold": 0.80,
8     "smoothing_window": 10,
9 }
10
11 criterion = nn.CrossEntropyLoss(weight=class_weights)
12 optimizer = optim.AdamW(model.parameters(), lr=CONFIG["learning_rate"]
13                        weight_decay=CONFIG["weight_decay"])
14 scheduler = optim.lr_scheduler.ReduceLROnPlateau(
15     optimizer, mode="max", factor=0.5, patience=3, min_lr=1e-6)
16
17 use_amp = torch.cuda.is_available()
18 scaler = GradScaler(enabled=use_amp)
19
20 print("Training config:")
21 for k, v in CONFIG.items(): print(f" {k}: {v}")
22 print(f"Loss: CrossEntropyLoss (weighted)")
23 print(f"Optimizer: AdamW, Scheduler: ReduceLROnPlateau")
24 print(f"Mixed precision: {use_amp}")

```

```

Training config:
epochs: 10
learning_rate: 0.0003
weight_decay: 0.0001
patience: 5
min_delta: 0.001
confidence_threshold: 0.8
smoothing_window: 10
Loss: CrossEntropyLoss (weighted)
Optimizer: AdamW, Scheduler: ReduceLROnPlateau
Mixed precision: True

```

✓ Section 18: Model Training

Training with validation, early stopping, best model saving, and mixed precision.

```

1 def train_one_epoch(model, loader, criterion, optimizer, device, scaler):
2     model.train()
3     running_loss = 0.0; correct = 0; total = 0
4     for images, labels in tqdm(loader, desc="Training", leave=False):
5         images, labels = images.to(device), labels.to(device)
6         optimizer.zero_grad()
7         if use_amp:
8             with autocast():
9                 outputs = model(images); loss = criterion(outputs, labels)
10                scaler.scale(loss).backward(); scaler.step(optimizer); scaler.update()
11        else:
12            outputs = model(images); loss = criterion(outputs, labels)
13            loss.backward(); optimizer.step()
14            running_loss += loss.item() * images.size(0)
15            _, pred = outputs.max(1); total += labels.size(0)
16            correct += pred.eq(labels).sum().item()
17    return running_loss / total, 100.0 * correct / total
18
19 def validate(model, loader, criterion, device):
20     model.eval()
21     running_loss = 0.0; correct = 0; total = 0
22     all_preds, all_labels = [], []
23     with torch.no_grad():
24         for images, labels in tqdm(loader, desc="Validating", leave=False):
25             images, labels = images.to(device), labels.to(device)
26             outputs = model(images); loss = criterion(outputs, labels)
27             running_loss += loss.item() * images.size(0)
28             _, pred = outputs.max(1); total += labels.size(0)
29             correct += pred.eq(labels).sum().item()
30             all_preds.extend(pred.cpu().numpy())
31             all_labels.extend(labels.cpu().numpy())
32    return running_loss / total, 100.0 * correct / total, all_preds, all_labels

```

```

1 import torch
2 import gc
3
4 def train_model(model, train_loader, val_loader, criterion, optimizer, scheduler, device, config):
5     best_val = 0.0; best_state = None; patience_ctr = 0
6     history = {"epoch": [], "train_loss": [], "train_acc": [],
7               "val_loss": [], "val_acc": [], "lr": []}
8     print(f"\nTraining for {config['epochs']} epochs (patience={config['patience']})")
9     print("=" * 60)
10    t0 = time.time()
11
12    if use_amp:
13        from torch.cuda.amp import GradScaler
14        scaler = GradScaler()
15    else:
16        scaler = None

```



```
17
18     for epoch in range(1, cfg["epochs"] + 1):
19         ep0 = time.time()
20
21         if dev.type == 'cuda':
22             torch.cuda.empty_cache()
23             gc.collect()
24
25         model.train()
26         t_loss, t_acc = train_one_epoch(model, train_ldr, crit, opt,
27
28         model.eval()
29         with torch.no_grad():
30             v_loss, v_acc, _, _ = validate(model, val_ldr, crit, dev
31
32         sched.step(v_acc)
33         lr = opt.param_groups[0]["lr"]
34
35         history["epoch"].append(epoch)
36         history["train_loss"].append(t_loss); history["train_acc"].a
37         history["val_loss"].append(v_loss); history["val_acc"].appen
38         history["lr"].append(lr)
39
40         print(f"Epoch [{epoch:3d}/{cfg['epochs']}] Train: {t_loss:.4
41
42         if v_acc > best_val + cfg["min_delta"]:
43             best_val = v_acc
44             best_state = copy.deepcopy(model.state_dict())
45             patience_ctr = 0
46             torch.save({"epoch": epoch, "model_state_dict": model.st
47                         "optimizer_state_dict": opt.state_dict(), "v
48                         DIRS["models"] / "best_model.pth")
49             print(f" >> New best: {v_acc:.2f}%")
50         else:
51             patience_ctr += 1
52
53         if patience_ctr >= cfg["patience"]:
54             print(f"\nEarly stopping at epoch {epoch}. Best: {best_v
55             break
56
57         torch.save({"epoch": epoch, "model_state_dict": model.state_dict
58                     "optimizer_state_dict": opt.state_dict(), "val_accu
59                     "config": cfg, "class_mapping": class_mapping, "reve
60                     DIRS["models"] / "final_model.pth")
61
62         if best_state is not None:
63             model.load_state_dict(best_state)
64
65         pd.DataFrame(history).to_csv(DIRS["results"] / "training_history
66         print(f"\nTraining done in {time.time()-t0:.1f}s. Best val acc:
67
68         if dev.type == 'cuda':
69             torch.cuda.empty_cache()
70
```

```

71     return model, history, best_val
72
73 # 1. Determine actual class count and validate labels
74 actual_num_classes = train_df['label_id'].max() + 1
75 print(f"Adjusting for {actual_num_classes} detected classes...")
76
77 # CRITICAL: Verify all labels are within range [0, actual_num_classes)
78 if train_df['label_id'].min() < 0 or train_df['label_id'].max() >= actual_num_classes:
79     raise ValueError(f"Labels in train_df are out of bounds: {train_df['label_id'].min():.0f} to {train_df['label_id'].max():.0f}")
80
81 # 2. Re-initialize model
82 if torch.cuda.is_available():
83     torch.cuda.empty_cache()
84     gc.collect()
85
86 model = create_efficientnet_b0(actual_num_classes, pretrained=True)
87
88 # 3. Re-initialize optimizer/scheduler
89 optimizer = optim.AdamW(model.parameters(), lr=CONFIG["learning_rate"])
90 scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode="max", patience=10)
91
92 # 4. Re-compute weights and criterion
93 class_weights = compute_class_weights(train_dataset)
94 criterion = nn.CrossEntropyLoss(weight=class_weights)
95
96 # 5. Start training
97 try:
98     model, training_history, best_val_acc = train_model(
99         model, train_loader, val_loader, criterion, optimizer, scheduler, patience=10
100 )
101 except RuntimeError as e:
102     print(f"An error occurred during training: {e}")
103     torch.cuda.empty_cache()
104     gc.collect()

```

Adjusting for 7 detected classes...

Loaded pretrained weights for efficientnet-b0

Training for 10 epochs (patience=5)...

=====

```

Epoch [ 1/10] Train: 0.1327/96.77% | Val: 0.0037/99.88% | LR: 3.00e-04 |
>> New best: 99.88%
Epoch [ 2/10] Train: 0.0148/99.61% | Val: 0.0078/99.81% | LR: 3.00e-04 |
Epoch [ 3/10] Train: 0.0077/99.76% | Val: 0.0036/99.88% | LR: 3.00e-04 |
Epoch [ 4/10] Train: 0.0044/99.91% | Val: 0.0051/99.88% | LR: 3.00e-04 |
Epoch [ 5/10] Train: 0.0059/99.85% | Val: 0.0042/99.88% | LR: 1.50e-04 |
Epoch [ 6/10] Train: 0.0032/99.90% | Val: 0.0007/100.00% | LR: 1.50e-04 |
>> New best: 100.00%
Epoch [ 7/10] Train: 0.0015/99.95% | Val: 0.0003/100.00% | LR: 1.50e-04 |
Epoch [ 8/10] Train: 0.0036/99.90% | Val: 0.0003/100.00% | LR: 1.50e-04 |
Epoch [ 9/10] Train: 0.0016/99.93% | Val: 0.0009/99.96% | LR: 1.50e-04 |
Epoch [ 10/10] Train: 0.0007/99.97% | Val: 0.0003/100.00% | LR: 7.50e-05 |

```

Training done in 1167.0s. Best val acc: 100.00%

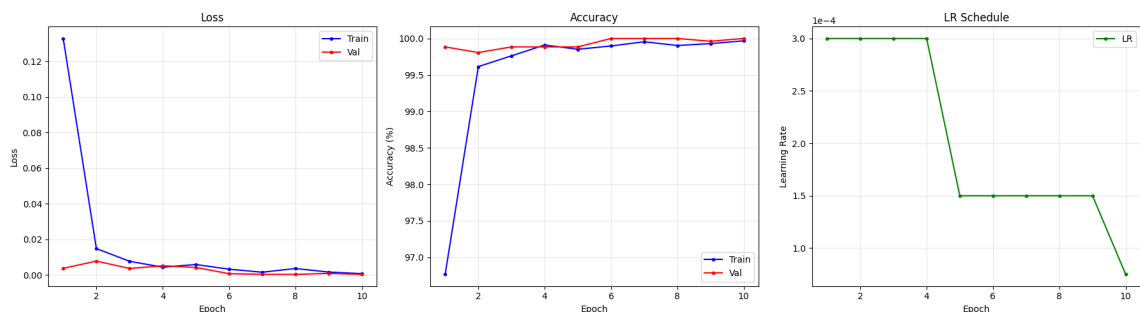
Section 10: Training Curves

SECTION 19: TRAINING CURVES

```

1 def plot_training_curves(hist, save_path):
2     fig, axes = plt.subplots(1, 3, figsize=(18, 5))
3     axes[0].plot(hist["epoch"], hist["train_loss"], "b-o", label="Train")
4     axes[0].plot(hist["epoch"], hist["val_loss"], "r-o", label="Val",
5     axes[0].set_xlabel("Epoch"); axes[0].set_ylabel("Loss")
6     axes[0].set_title("Loss"); axes[0].legend(loc="best"); axes[0].gr
7
8     axes[1].plot(hist["epoch"], hist["train_acc"], "b-o", label="Train")
9     axes[1].plot(hist["epoch"], hist["val_acc"], "r-o", label="Val",
10    axes[1].set_xlabel("Epoch"); axes[1].set_ylabel("Accuracy (%)")
11    axes[1].set_title("Accuracy"); axes[1].legend(loc="best"); axes[1]
12
13    axes[2].plot(hist["epoch"], hist["lr"], "g-o", label="LR", marker
14    axes[2].set_xlabel("Epoch"); axes[2].set_ylabel("Learning Rate")
15    axes[2].set_title("LR Schedule"); axes[2].legend(loc="best"); ax
16    axes[2].ticklabel_format(style="sci", axis="y", scilimits=(0, 0))
17    plt.tight_layout()
18    plt.savefig(save_path, dpi=150, bbox_inches="tight"); plt.show()
19    print(f"Saved: {save_path}")
20
21 plot_training_curves(training_history, DIRS["results"] / "training_cu

```



Saved: /content/BSL_Project/results/training_curves.png

Section 20: Model Evaluation

```

1 # Load best model
2 ckpt = torch.load(DIRS["models"] / "best_model.pth", map_location=dev
3 model.load_state_dict(ckpt["model_state_dict"])
4 print(f"Loaded best model from epoch {ckpt['epoch']} (val_acc={ckpt['
5

```

```

6 test_loss, test_acc, all_preds, all_labels = validate(model, test_loader)
7 print(f"\nTest Loss: {test_loss:.4f}, Test Accuracy: {test_acc:.2f}%")

```

Loaded best model from epoch 6 (val_acc=100.00%)

Test Loss: 0.0003, Test Accuracy: 100.00%

```

1 target_names = [reverse_mapping[i] for i in range(len(reverse_mapping))]
2 present = sorted(set(all_labels))
3 present_names = [target_names[i] for i in present]
4
5 report = classification_report(all_labels, all_preds, labels=present,
6                               target_names=present_names, digits=4,
7
8 report_path = DIRS["results"] / "classification_report.txt"
9 with open(report_path, "w") as f:
10     f.write("BSL SIGN RECOGNITION - CLASSIFICATION REPORT\n")
11     f.write("=" * 60 + "\n\n")
12     f.write(f"Model: EfficientNet-B0 | Test Accuracy: {test_acc:.2f}%")
13     f.write(f>Date: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}\n\n")
14     f.write(classification_report(all_labels, all_preds, labels=present,
15                                 target_names=present_names, digits=4,
16
17 print(classification_report(all_labels, all_preds, labels=present,
18                             target_names=present_names, digits=4))
19 print(f"Report saved: {report_path}")
20
21 overall_acc = report["accuracy"]
22 macro_f1 = report["macro avg"]["f1-score"]
23 print(f"\nAccuracy: {overall_acc:.4f}, Macro F1: {macro_f1:.4f}")

```

	precision	recall	f1-score	support
A	1.0000	1.0000	1.0000	376
B	1.0000	1.0000	1.0000	376
C	1.0000	1.0000	1.0000	377
D	1.0000	1.0000	1.0000	377
E	1.0000	1.0000	1.0000	377
F	1.0000	1.0000	1.0000	334
G	1.0000	1.0000	1.0000	376
accuracy			1.0000	2593
macro avg	1.0000	1.0000	1.0000	2593
weighted avg	1.0000	1.0000	1.0000	2593

Report saved: /content/BSL_Project/results/classification_report.txt

Accuracy: 1.0000, Macro F1: 1.0000

```

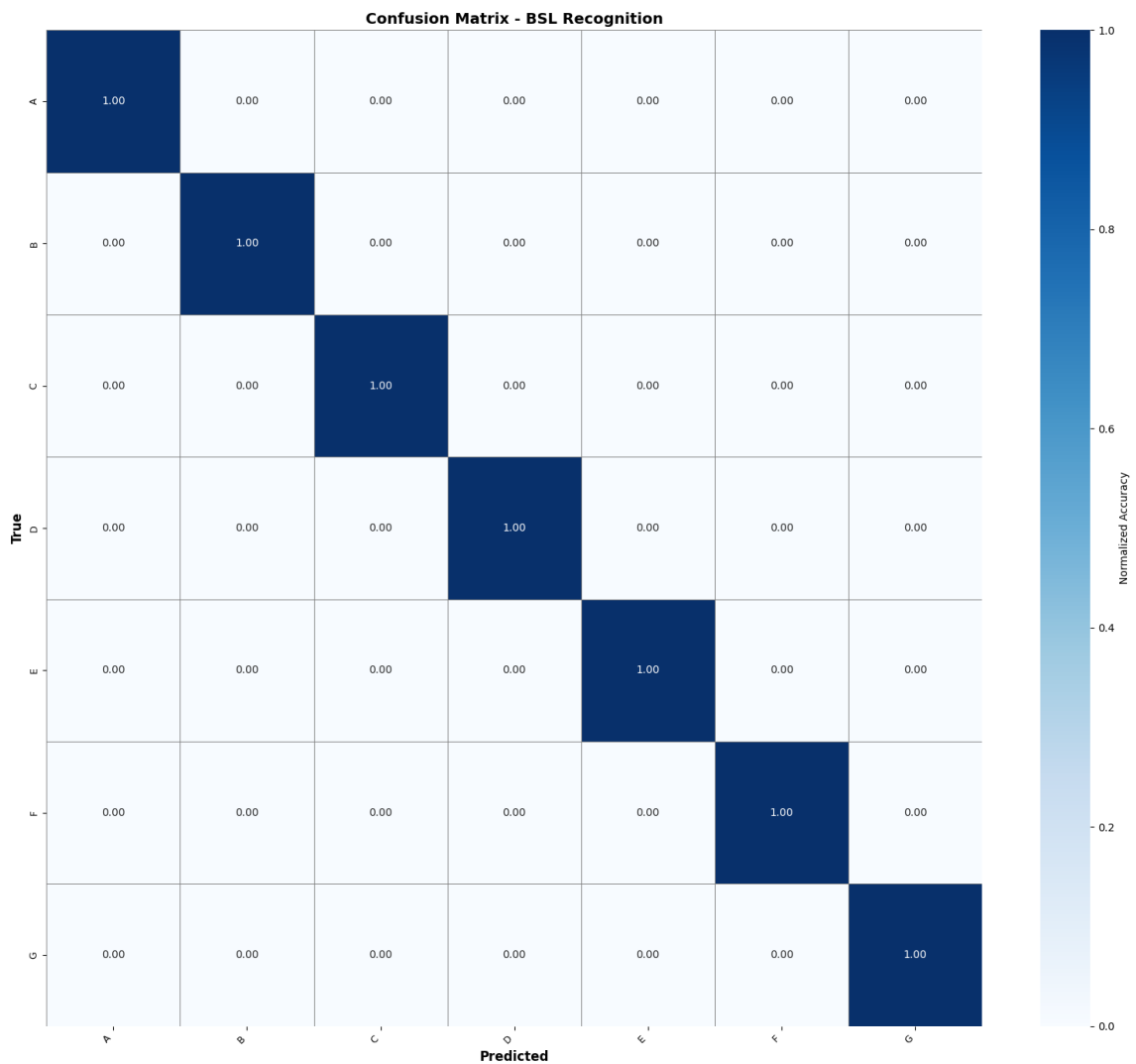
1 def plot_confusion_matrix(true, pred, names, save_path):
2     cm = confusion_matrix(true, pred, labels=range(len(names)))
3     cm_norm = cm.astype(float) / cm.sum(axis=1)[:, np.newaxis]
4     fig, ax = plt.subplots(figsize=(16, 14))
5     sns.heatmap(cm_norm, annot=True, fmt=".2f", cmap="Blues",
6                 xticklabels=names, yticklabels=names, ax=ax)

```

```

7         cbar_kws={"label": "Normalized Accuracy"},
8         linewidths=0.5, linecolor="gray")
9     ax.set_xlabel("Predicted", fontsize=12, fontweight="bold")
10    ax.set_ylabel("True", fontsize=12, fontweight="bold")
11    ax.set_title("Confusion Matrix - BSL Recognition", fontsize=14, f
12    plt.xticks(rotation=45, ha="right", fontsize=9)
13    plt.yticks(fontsize=9)
14    plt.tight_layout()
15    plt.savefig(save_path, dpi=150, bbox_inches="tight"); plt.show()
16    print(f"Saved: {save_path}")
17    return cm
18
19 conf_matrix = plot_confusion_matrix(all_labels, all_preds, target_nam
20                                     DIRS["results"] / "confusion_matr

```



Saved: /content/BSL_Project/results/confusion_matrix.png

```

1 # Per-class accuracy
2 per_class = {}
3 for cid in present:
4     mask = np.array(all_labels) == cid
5     correct = (np.array(all_preds)[mask] == cid).sum()
6     total = mask.sum()
7     per_class[target_names[cid]] = {"accuracy": correct / total if to
8                                     "correct": int(correct), "total"
9 sorted_acc = sorted(per_class.items(), key=lambda x: x[1]["accuracy"])
10 print("Per-class accuracy (sorted):")
11 print(f"  {'Class':<6} {'Acc':>8} {'Correct':>8} {'Total':>8}")
12 print("  " + "-" * 34)
13 for name, s in sorted_acc:
14     print(f"  {name:<6} {s['accuracy']:>8.2%} {s['correct']:>8} {s['t
15 print(f"\nBest: {sorted_acc[0][0]} ({sorted_acc[0][1]['accuracy']:.2%
16 print(f"Worst: {sorted_acc[-1][0]} ({sorted_acc[-1][1]['accuracy']:.2%

```

Per-class accuracy (sorted):

Class	Acc	Correct	Total
A	100.00%	376	376
B	100.00%	376	376
C	100.00%	377	377
D	100.00%	377	377
E	100.00%	377	377
F	100.00%	334	334
G	100.00%	376	376

Best: A (100.00%)

Worst: G (100.00%)

Section 21: Test Predictions Export

```

1 def save_test_predictions(model, loader, dev, rmap, save_path):
2     model.eval(); results = []
3     with torch.no_grad():
4         for imgs, lbls in tqdm(loader, desc="Predicting"):
5             imgs = imgs.to(dev)
6             out = model(imgs); prob = torch.softmax(out, dim=1)
7             conf, pred = prob.max(dim=1)
8             for i in range(imgs.size(0)):
9                 results.append({"label_id": int(lbls[i]), "predicted_
10                             "confidence": float(conf[i]),
11                             "label_name": rmap[int(lbls[i])],
12                             "predicted_name": rmap[int(pred[i])],
13                             "correct": int(lbls[i]) == int(pred[i])})
14     df = pd.DataFrame(results); df.to_csv(save_path, index=False)
15     print(f"Predictions saved: {save_path} ({df['correct'].mean():.2%})")
16     return df
17
18 # Note: Using reverse_mapping here to correctly translate numeric IDs
19 pred_df = save_test_predictions(model, test_loader, device, reverse_m
20                               DIRS["results"] / "test_predictions.c

```

Predicting: 100%|██████████| 82/82 [00:07<00:00, 11.66it/s]

Predictions saved: /content/BSL_Project/results/test_predictions.csv (100.0%)

Section 22: Misclassification Analysis

```

1 def analyze_misclassifications(dataset, model, dev, rmap, save_path):
2     model.eval(); misclassified = []
3     with torch.no_grad():
4         for idx in tqdm(range(len(dataset)), desc="Checking"):
5             img, lbl = dataset[idx]
6             out = model(img.unsqueeze(0).to(dev))
7             prob = torch.softmax(out, dim=1); conf, pred = prob.max(dim=1)
8             if pred.item() != lbl:
9                 row = dataset.df.iloc[idx]
10                misclassified.append({"image_path": row.get("image_path"),
11                                    "crop_path": row.get("crop_path"),
12                                    "true_label": rmap[lbl],
13                                    "predicted_label": rmap[pred.item()],
14                                    "confidence": round(conf.item(), 4)})
15     df = pd.DataFrame(misclassified); df.to_csv(save_path, index=False)
16     print(f"Misclassified: {len(df)}")
17     if len(df) > 0:
18         print(f"Avg confidence on errors: {df['confidence'].mean():.4}")
19         pairs = df.groupby(["true_label", "predicted_label"]).size().reset()
20         print("\nMost common confusions:")

```

```

21         for (t, p), c in pairs.head(10).items():
22             print(f" {t} -> {p}: {c}")
23     return df
24
25 misclf_df = analyze_misclassifications(test_dataset, model, device, r
26                                     DIRS["results"] / "misclassifi

```

Checking: 100%|██████████| 2593/2593 [00:38<00:00, 68.08it/s]Misclassified

```

1 def visualize_misclassified(dataset, misclf_df, cmap, max_s=12):
2     if len(misclf_df) == 0: print("No misclassified samples!"); retur
3     ncols = 4; samples = misclf_df.sample(n=min(max_s, len(misclf_df)
4     nrows = (len(samples) + ncols - 1) // ncols
5     fig, axes = plt.subplots(nrows, ncols, figsize=(16, 4*nrows))
6
7     # Ensure axes is a flat array even if it's a single axis
8     if nrows * ncols == 1:
9         axes = np.array([axes])
10    else:
11        axes = axes.flatten()
12
13    for idx, (_, r) in enumerate(samples.iterrows()):
14        ax = axes[idx]
15        p = Path(r["crop_path"]) if r.get("crop_path") and Path(r["cr
16        if p.exists():
17            ax.imshow(Image.open(p).convert("RGB"))
18            ax.set_title(f"True: {r['true_label']} | Pred: {r['predicted_
19                        fontsize=9, color="red", fontweight="bold")
20            ax.axis("off")
21
22    for i in range(len(samples), len(axes)):
23        axes[i].axis("off")
24
25    plt.suptitle("Misclassified Samples", fontsize=14, fontweight="bo
26    plt.tight_layout()
27    plt.savefig(DIRS["results"] / "misclassified_samples.png", dpi=15
28    plt.show()
29
30 visualize_misclassified(test_dataset, misclf_df, class_mapping)

```

No misclassified samples!

✓ Section 23: Real-Time Webcam Inference

Pipeline: Webcam frame > MediaPipe hand detection > Hand crop > Resize

224x224 > Normalize > Model prediction > Confidence threshold (0.80) > Prediction
smoothing (majority vote, last 10 frames) > Display

```
1 import mediapipe as mp
```



```

2 from mediapipe.tasks import python
3 from mediapipe.tasks.python import vision
4 from collections import Counter
5
6 # Reuse the model path from Section 8
7 model_path = 'hand_landmarker.task'
8
9 # Initialize the Tasks HandLandmarker for live stream mode
10 base_options = python.BaseOptions(model_asset_path=model_path)
11 options = vision.HandLandmarkerOptions(
12     base_options=base_options,
13     running_mode=vision.RunningMode.VIDEO,
14     num_hands=1,
15     min_hand_detection_confidence=0.7,
16     min_hand_presence_confidence=0.5,
17     min_tracking_confidence=0.5
18 )
19 webcam_hands_detector = vision.HandLandmarker.create_from_options(opt
20
21 CONF_THRESH = CONFIG["confidence_threshold"]
22 SMOOTH_WIN = CONFIG["smoothing_window"]
23
24 def preprocess_for_model(img_bgr):
25     rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
26     pil = Image.fromarray(rgb)
27     t = val_transform(pil)
28     return t.unsqueeze(0)
29
30 def smooth_prediction(label, conf, buf, win=SMOOTH_WIN):
31     buf.append(label if conf >= CONF_THRESH else "uncertain")
32     if len(buf) > win: buf.pop(0)
33     return Counter(buf).most_common(1)[0][0]

```

```

1 def run_realtime_inference(model, cmap, rmap, dev, cam_idx=0):
2     print("Starting real-time BSL recognition... Press q to quit")
3     cap = cv2.VideoCapture(cam_idx)
4     if not cap.isOpened():
5         print("Cannot open webcam."); return None
6     cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
7     cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
8     model.eval(); buf = []; t0 = time.time(); fc = 0; inf_times = []
9
10    while True:
11        ret, frame = cap.read()
12        if not ret: break
13        fc += 1; frame_start = time.time()
14        frame = cv2.flip(frame, 1) # Mirror
15        rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
16
17        # Convert to MediaPipe Image for Tasks API
18        mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=rg
19        # Using detection for video mode requires a timestamp
20        timestamp_ms = int((time.time() - t0) * 1000)
21        res = webcam_hands_detector.detect_for_video(mp_image, timest

```

```

21     res = webcam_handus_detector.detect_for_video(mp_image, times)
22
23     pred_text = "No hand detected"; conf = 0.0; color = (0, 0, 255)
24     if res.hand_landmarks:
25         hl = res.hand_landmarks[0]; h, w = frame.shape[:2]
26         xc = [lm.x*w for lm in hl]; yc = [lm.y*h for lm in hl]
27         x1 = max(0, int(min(xc))-20); y1 = max(0, int(min(yc))-20)
28         x2 = min(w, int(max(xc))+20); y2 = min(h, int(max(yc))+20)
29         crop = frame[y1:y2, x1:x2]
30         if crop.size > 0:
31             inp = preprocess_for_model(crop).to(dev)
32             with torch.no_grad():
33                 out = model(inp); prob = torch.softmax(out, dim=1)
34                 c, p = prob.max(dim=1)
35                 conf = c.item(); raw = rmap[p.item()]
36                 smoothed = smooth_prediction(raw, conf, buf, SMOOTH_W)
37                 pred_text = "Uncertain" if smoothed == "uncertain" else
38                 color = (0, 255, 0) if pred_text != "Uncertain" else
39                 inf_times.append(time.time() - frame_start)
40             cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
41
42     fps = fc / (time.time() - t0)
43     display = f"BSL: {pred_text}" + (f" ({conf:.1%})" if pred_text
44     cv2.rectangle(frame, (10, 10), (380, 80), (0, 0, 0), -1)
45     cv2.putText(frame, display, (20, 40), cv2.FONT_HERSHEY_SIMPLE,
46     cv2.putText(frame, f"FPS: {fps:.1f}", (20, 65), cv2.FONT_HERSHEY
47
48     # In Colab, cv2.imshow won't work normally; users typically u
49     # But we keep this for local/standard Jupyter compatibility
50     try:
51         cv2.imshow("BSL Sign Recognition", frame)
52         if cv2.waitKey(1) & 0xFF == ord("q"): break
53     except Exception:
54         # Fallback for headless environments if necessary
55         break
56
57     cap.release(); cv2.destroyAllWindows()
58     avg_fps = fc / (time.time() - t0) if fc else 0
59     avg_inf = np.mean(inf_times) * 1000 if inf_times else 0
60     perf = {"average_fps": round(avg_fps, 2), "avg_inference_ms": rou
61            "total_frames": fc, "duration_s": round(time.time()-t0, 2)
62     print(f"\nSession ended. FPS: {avg_fps:.1f}, Inference: {avg_inf:
63     return perf
64
65 print("Webcam inference (Tasks API) ready.")

```

Webcam inference (Tasks API) ready.

✓ Section 24: Real-Time Performance Recording

```

1 perf_path = DIRS["results"] / "realtime_performance.txt"
2 template = f"""Real-Time BSL Recognition Performance
-

```

```

3 =====
4 Date: {datetime.now().strftime("%Y-%m-%d")}
5 Tester: [Fill in]
6 GPU: {gpu_name if torch.cuda.is_available() else "CPU only"}
7
8 --- Metrics ---
9 Frames Per Second: [Fill in]
10 Average Inference (ms): [Fill in]
11 Confidence Threshold: {CONF_THRESH}
12 Smoothing Window: {SMOOTH_WIN}
13
14 --- Conditions ---
15 Lighting: [Bright/Dim/Outdoor]
16 Background: [Plain/Cluttered]
17 Hand Position: [Center/Off-center]
18
19 --- Feedback ---
20 Ease of Use (1-10):
21 Recognition Speed (1-10):
22 Accuracy Impression (1-10):
23 Notes: [Free text]
24 ""
25 with open(perf_path, "w") as f: f.write(template)
26 print(f"Performance template saved: {perf_path}")

```

Performance template saved: /content/BSL_Project/results/realtime_performa

✓ Section 25: User Testing Protocol

```

1 ut_path = DIRS["results"] / "user_testing_protocol.txt"
2 ut = f"""BSL User Testing Protocol
3 =====
4
5 Session ID: [Auto]
6 Date: {datetime.now().strftime("%Y-%m-%d")}
7 User: user_test_[N]
8 BSL Experience: [None/Beginner/Intermediate/Fluent]
9
10 --- Environment ---
11 Lighting: [Bright/Dim/Outdoor]
12 Background: [Plain/Cluttered]
13 Camera: [Fill in]
14
15 --- Test Results ---
16 Sign | Correct | Predicted | Confidence | Notes
17 -----|-----|-----|-----|-----
18 A | | | |
19 B | | | |
20 [... continue for all 26 letters ...]
21 Z | | | |
22
23 --- Summary ---

```

```

24 Total: 26
25 Correct: [N]
26 Accuracy: [N/26]
27 Most Confused: [List]
28
29 --- Feedback ---
30 Q1: Ease of use (1-10):
31 Q2: Speed (1-10):
32 Q3: Accuracy impression (1-10):
33 Q4: Would you use daily? (Y/N):
34 Q5: Suggestions:
35 ""
36 with open(ut_path, "w") as f: f.write(ut)
37 print(f"User testing template: {ut_path}")

```

User testing template: /content/BSL_Project/results/user_testing_protocol.

✓ Section 26: Limitations

```

1 lim_path = DIRS["results"] / "limitations.txt"
2 limitations = [
3     "LIMITATIONS OF BSL SIGN RECOGNITION PROTOTYPE (v1.0)",
4     "=" * 60,
5     "",
6     "1. ISOLATED SIGN RECOGNITION ONLY",
7     "    Recognises isolated static BSL signs from single images.",
8     "    Does NOT perform continuous sign language recognition or",
9     "    translate BSL grammar. Real BSL involves sequential gestures,",
10    "    co-articulation, and grammatical structures differing from En
11    "",
12    "2. LIMITED VOCABULARY",
13    "    Only BSL fingerspelling alphabet (A-Z). Does NOT recognise",
14    "    BSL numbers, common words, greeting signs, or two-hand signs.
15    "",
16    "3. STATIC IMAGE CONSTRAINTS",
17    "    Trained on static images, not video. Signs relying on motion"
18    "    (YES, NO, HELLO) cannot be reliably recognised.",
19    "",
20    "4. NO FACIAL EXPRESSION / BODY POSTURE ANALYSIS",
21    "    BSL grammar relies heavily on facial expressions, mouth patte
22    "    head tilt, shoulder position. None of these are captured.",
23    "",
24    "5. LIGHTING AND ENVIRONMENT DEPENDENCY",
25    "    Degrades in poor lighting, cluttered backgrounds may interfer
26    "    assumes single clearly visible hand.",
27    "",
28    "6. CAMERA QUALITY DEPENDENCY",
29    "    Low-resolution cameras produce blurry crops. Different camera
30    "    may produce different colour profiles affecting detection.",
31    "",
32    "7. HAND VISIBILITY REQUIREMENTS",
33    "    Hand must be fully visible. Partial occlusion (gloves, sleeve

```

```

34     "    may fail. Skin tone representation in training data matters."
35     "",
36     "8. CONFUSION BETWEEN SIMILAR SIGNS",
37     "    Some BSL letters look similar (U/V, M/N, D and G/I).",
38     "",
39     "9. LIMITED GENERALISATION",
40     "    May not generalise to unseen users, signing speeds, hand size
41     "    personal variations, or regional BSL variations.",
42     "",
43     "10. NO SENTENCE-LEVEL CONTEXT",
44     "    No understanding of context or BSL sentence construction.",
45     "",
46     "11. REAL-TIME PERFORMANCE LIMITATIONS",
47     "    FPS depends on GPU. Latency may be unacceptable for fluid",
48     "    conversation. MediaPipe adds computational overhead.",
49     "",
50     "12. SAFETY CONCERNS",
51     "    NOT for sole communication. Research prototype only.",
52     "    Incorrect translations could cause misunderstandings.",
53 ]
54 with open(lim_path, "w") as f: f.write("\n".join(limitations))
55 print(f"Limitations saved: {lim_path}")
56 for line in limitations[:20]: print(line)

```

```

Limitations saved: /content/BSL_Project/results/limitations.txt
LIMITATIONS OF BSL SIGN RECOGNITION PROTOTYPE (v1.0)
=====

```

1. ISOLATED SIGN RECOGNITION ONLY
Recognises isolated static BSL signs from single images.
Does NOT perform continuous sign language recognition or
translate BSL grammar. Real BSL involves sequential gestures,
co-articulation, and grammatical structures differing from English.
2. LIMITED VOCABULARY
Only BSL fingerspelling alphabet (A-Z). Does NOT recognise
BSL numbers, common words, greeting signs, or two-hand signs.
3. STATIC IMAGE CONSTRAINTS
Trained on static images, not video. Signs relying on motion
(YES, NO, HELLO) cannot be reliably recognised.
4. NO FACIAL EXPRESSION / BODY POSTURE ANALYSIS
BSL grammar relies heavily on facial expressions, mouth patterns,
head tilt, shoulder position. None of these are captured.

✓ Section 27: Future Improvement Plan

```

1 fut_path = DIRS["results"] / "future_improvement_plan.txt"
2 future = [
3     "FUTURE IMPROVEMENT PLAN",
4     "=" * 50,
5     "",
6     "PHASE 1: DATA (1-3 months)"

```

```

6     "PHASE 1: DATA (1-3 months)",
7     "- Add BSL numbers, greetings, common words (target: 50-100 signs)",
8     "- Collect from 50+ users with varied demographics",
9     "- Multiple lighting, background, and camera conditions",
10    "- Record video clips (2-5s) for dynamic signs",
11    "",
12    "PHASE 2: MODEL (3-6 months)",
13    "- CNN-LSTM for spatial + temporal features",
14    "- 3D CNN for spatiotemporal pattern recognition",
15    "- Vision Transformers for video understanding",
16    "- Facial expression recognition (MediaPipe Face Mesh)",
17    "- Head pose and body posture tracking",
18    "- Advanced augmentation: CutMix, MixUp, GAN synthetic data",
19    "- Model optimization: distillation, quantization, NAS",
20    "",
21    "PHASE 3: SYSTEM (3-6 months)",
22    "- Continuous sign recognition with sliding window",
23    "- Sign segmentation and boundary detection",
24    "- Context-aware recognition with language model",
25    "- Text-to-speech output integration",
26    "",
27    "PHASE 4: DEPLOYMENT (6-12 months)",
28    "- Web app: FastAPI backend + React frontend",
29    "- Mobile app: React Native / Flutter + TFLite",
30    "- On-device inference for offline use",
31    "- Cloud auto-scaling and monitoring",
32    "- A/B testing framework",
33    "",
34    "PHASE 5: RESEARCH (ongoing)",
35    "- User studies with deaf BSL community",
36    "- Bias audit across demographics",
37    "- Privacy protection for video data",
38    "- Publish at CVPR/ICCV/ECCV",
39    "- Open-source dataset and models",
40 ]
41 with open(fut_path, "w") as f: f.write("\n".join(future))
42 print(f"Future plan saved: {fut_path}")
43 for line in future: print(line)

```

```

Future plan saved: /content/BSL_Project/results/future_improvement_plan.tx
FUTURE IMPROVEMENT PLAN
=====

```

PHASE 1: DATA (1-3 months)

- Add BSL numbers, greetings, common words (target: 50-100 signs)
- Collect from 50+ users with varied demographics
- Multiple lighting, background, and camera conditions
- Record video clips (2-5s) for dynamic signs

PHASE 2: MODEL (3-6 months)

- CNN-LSTM for spatial + temporal features
- 3D CNN for spatiotemporal pattern recognition
- Vision Transformers for video understanding
- Facial expression recognition (MediaPipe Face Mesh)
- Head pose and body posture tracking
- Advanced augmentation: CutMix, MixUp, GAN synthetic data

- Model optimization: distillation, quantization, NAS

PHASE 3: SYSTEM (3-6 months)

- Continuous sign recognition with sliding window
- Sign segmentation and boundary detection
- Context-aware recognition with language model
- Text-to-speech output integration

PHASE 4: DEPLOYMENT (6-12 months)

- Web app: FastAPI backend + React frontend
- Mobile app: React Native / Flutter + TFLite
- On-device inference for offline use
- Cloud auto-scaling and monitoring
- A/B testing framework

PHASE 5: RESEARCH (ongoing)

- User studies with deaf BSL community
- Bias audit across demographics
- Privacy protection for video data
- Publish at CVPR/ICCV/ECCV
- Open-source dataset and models

✓ Section 28: Project Summary

```

1 print("=" * 60)
2 print("PROJECT DELIVERABLES")
3 print("=" * 60)
4
5 deliverables = {
6     "Annotations": [
7         "annotations.csv", "class_mapping.json", "final_annotations.c
8         "hand_annotations.csv", "validation_log.csv", "review_report.
9     ],
10    "Models": ["best_model.pth", "final_model.pth"],
11    "Results": [
12        "training_history.csv", "training_curves.png",
13        "classification_report.txt", "confusion_matrix.png",
14        "test_predictions.csv", "misclassified_samples.csv",
15        "misclassified_samples.png", "flagged_images_review.png",
16        "realtime_performance.txt", "user_testing_protocol.txt",
17        "limitations.txt", "future_improvement_plan.txt",
18    ],
19 }
20
21 for cat, items in deliverables.items():
22     print(f"\n{cat}:")
23     for item in items:
24         if cat == "Annotations":
25             p = DIRS["annotations"] / item
26         elif cat == "Models":
27             p = DIRS["models"] / item
28         else:
29             p = DIRS["results"] / item
30         exists = "OK" if p.exists() else "PENDING"

```

```

31     print(f"  [{exists}] {p}")
32
33 print(f"\nModel: EfficientNet-B0")
34 print(f"Train: {len(train_dataset)}, Val: {len(val_dataset)}, Test: {
35 print(f"Best Val Acc: {best_val_acc:.2f}%, Test Acc: {test_acc:.2f}%,

```

PROJECT DELIVERABLES

Annotations:

```

[OK] /content/BSL_Project/annotations/annotations.csv
[OK] /content/BSL_Project/annotations/class_mapping.json
[OK] /content/BSL_Project/annotations/final_annotations.csv
[OK] /content/BSL_Project/annotations/hand_annotations.csv
[OK] /content/BSL_Project/annotations/validation_log.csv
[OK] /content/BSL_Project/annotations/review_report.csv

```

Models:

```

[OK] /content/BSL_Project/models/best_model.pth
[OK] /content/BSL_Project/models/final_model.pth

```

Results:

```

[OK] /content/BSL_Project/results/training_history.csv
[OK] /content/BSL_Project/results/training_curves.png
[OK] /content/BSL_Project/results/classification_report.txt
[OK] /content/BSL_Project/results/confusion_matrix.png
[OK] /content/BSL_Project/results/test_predictions.csv
[OK] /content/BSL_Project/results/misclassified_samples.csv
[PENDING] /content/BSL_Project/results/misclassified_samples.png
[OK] /content/BSL_Project/results/flagged_images_review.png
[OK] /content/BSL_Project/results/realtime_performance.txt
[OK] /content/BSL_Project/results/user_testing_protocol.txt
[OK] /content/BSL_Project/results/limitations.txt
[OK] /content/BSL_Project/results/future_improvement_plan.txt

```

Model: EfficientNet-B0

Train: 15556, Val: 2593, Test: 2593

Best Val Acc: 100.00%, Test Acc: 100.00%, Macro F1: 1.0000

```

1 import zipfile
2
3 def zip_dir(directory, output_zip):
4     d = Path(directory)
5     with zipfile.ZipFile(output_zip, "w", zipfile.ZIP_DEFLATED) as z:
6         for f in d.rglob("*"):
7             if f.is_file():
8                 z.write(f, f.relative_to(d.parent))
9     sz = Path(output_zip).stat().st_size / (1024*1024)
10    print(f"  Zipped: {output_zip} ({sz:.1f} MB)")
11
12 print("Zipping results for download...")
13 zip_dir(DIRS["results"], PROJECT_ROOT / "BSL_Results.zip")
14 zip_dir(DIRS["models"], PROJECT_ROOT / "BSL_Models.zip")
15 zip_dir(DIRS["annotations"], PROJECT_ROOT / "BSL_Annotations.zip")
16

```



```
17 print("\nDone! All deliverables generated.")
```

```
Zippping results for download...
```

```
  Zipped: /content/BSL_Project/BSL_Results.zip (0.5 MB)
```

```
  Zipped: /content/BSL_Project/BSL_Models.zip (85.5 MB)
```

```
  Zipped: /content/BSL_Project/BSL_Annotations.zip (3.5 MB)
```

```
Done! All deliverables generated.
```

✓ Section 29: Quick Start Guide

Prerequisites:

1. Kaggle account with API token
2. Google Colab with GPU (Runtime > Change runtime type > T4 GPU)

Steps:

1. Run Section 1: Setup
2. Section 3: Upload kaggle.json, download dataset
3. Sections 4-12: Process data
4. Sections 13-18: Train model (~20-40 min on T4)
5. Sections 19-22: Evaluate
6. Section 23: Real-time inference (needs webcam)

Troubleshooting:

- **OOM:** Reduce batch_size to 16 or 8
- **Slow training:** Verify GPU with `torch.cuda.is_available()`
- **Kaggle fails:** Check API credentials